Headless Template Template Parameters

Document #:	P3158R0
Date:	2024-02-15
Project:	Programming Language C++
Audience:	Evolution

Reply-to:	James Touton <bekenn@gmail.com>

Contents

§ 1 Introduction

This paper introduces a new flavor of template template parameter that matches any template template argument, herein referred to as *headless* template template parameters because they are declared without a *template-head*. This feature is intended to be used with universal template parameters ([P1985R3], [P2989R0]), but may be adopted separately. This feature obsoletes a form of template argument matching that allows a template template parameter with a template parameter pack to match a template template argument without a template parameter pack (violating the contract implied by the form of the template template parameter). This paper also introduces constrained template template parameter declarations, as constraints are expected to be used frequently in combination with headless template template parameters.

§ 1.1 Universal Template Parameter Syntax

The syntax for the declaration of a universal template parameter is as yet undecided. [P1985R3] and [P2989R0] offer several possibilities, each of which has drawbacks associated with introducing a new keyword, repurposing an old keyword, designating an identifier as a contextual keyword, or some combination of the above. The examples in those papers are not consistent in their choice of syntax; for internal consistency, and to avoid the issues mentioned above, this paper will use `?` to declare a universal template parameter:

```
// Example from P2989R0
template <universal template>
constexpr bool is_variable = false;
template <auto a>
constexpr bool is_variable<a> = true;

// Same example, rewritten with ?
template <?>
constexpr bool is_variable = false;
template <auto a>
constexpr bool is_variable<a> = true;
```

§ 1.2 Universal Template Parameter Semantics

Taken together, [P1985R3] and [P2989R0] present several different models of universal template parameter semantics. For the purposes of this paper, we'll assume the semantics presented in [P2989R0]; that is:

— A universal template parameter binds to any template argument of any kind.

— A template that uses universal parameters may have partial specializations that refine the kind of a universal parameter.

— A universal template parameter may be used as a template argument for a template parameter of any kind; if, during instantiation, the substituted value is of a kind incompatible with the template parameter, the program is ill-formed.

— A universal template parameter may not be used in any other context.

§ 1.3 Universal Template Parameter Topics Not Covered

[P1985R3] discusses a number of additional hypothetical language features that would have interactions with universal template parameters where they to be adopted. As these are not currently part of C++, this paper will not discuss them. The features are:

— Concept template parameters

— Variable template parameters

— Universal aliases

§ 2 Motivation

The primary motivation for this feature is to make universal template parameters useful without backsliding on one of the key principles of [P0522R0], which is that the form of a template template parameter should be indicative of its proper use within its scope. A secondary motivation is to provide an alternative to an existing rule that violates this principle. These two issues are closely related.

§ 2.1 Template Template Argument Matching

[N2555] introduced a change to template template argument matching that allows a template template parameter with a template parameter pack to match a template template argument without a template parameter pack. This was done to grant the ability to write a metafunction that could accept a template and an arbitrary number of template arguments, and then apply the arguments to the template:

```
template <template <class...> class F, class... Args>
using apply = F<Args...>;

template <class T1>           struct A;
template <class T1, class T2> struct B;
template <class... Ts>        struct C;

template <class T> struct X { };

X<apply<A, int>> xa;                                  // OK
X<apply<B, int, float>> xb;                           // OK
X<apply<C, int, float, int, short, unsigned>> xc; // OK
```

This addressed the immediate need, but it did so in a way that undermines the contract implied by the *template-head* of the template template parameter. In the above example, it would be reasonable for the author of `apply` to expect that `F` is a template that can accept any number of type arguments, as is the case for `C`. However, `A` can only accept one argument, and `B` can only accept two arguments. The contract implied by the declaration of `F` is violated, but the parameter is allowed to bind anyway. If a metafunction actually does require that the template accept any number of arguments, then there is no longer any space in the existing syntax to express that constraint. We've taken syntax that ought to mean one thing and assigned an entirely different meaning to it.

[N2555] was a reasonable compromise for its time. It addressed a very real need at the cost of a small syntactic carve-out in a feature that nobody had seen before (variadic templates). Today, variadic templates are everywhere, template constraints permit us to make fine-grained decisions based on the content of a template declaration, and with reflection on the horizon, the ability to accurately express intent is more important than ever. If universal template parameters take an approach similar to [N2555], the damage will be much worse.

§ 2.2 Universal Template Parameters

In the example above, `apply` can only operate on templates taking some number of type arguments. If you wish to `apply` non-type or template arguments to a template that can

accept them, you're out of luck. There are two reasons for this. The first is that `F` is specified in such a way that it cannot bind to a template that can accept template or non-type arguments. The second is that `apply` itself is unable to accept template or non-type arguments in `Args`. Universal template parameters are an essential feature well-suited to solving *one* of those problems, but [P1985R3] attempts to solve both of those issues with the same feature by abusing template template parameter matching in much the same way that [N2555] did:

```
template <template <?...> typename F, ?... Args>
using apply = F<Args...>; // easy peasy!

// ok, r1 is std::array<int, 3>
using r1 = apply<std::array, int, 3>;
// ok, r2 is std::vector<int, std::pmr::allocator>
using r2 = apply<std::vector, int, std::pmr::allocator>;
```

As with [N2555], the parameter pack in the *template-head* of `F` is misleading. This much follows from the rules established by [N2555]; it is not a new problem, and we can't change the meaning of the parameter pack without a lengthy deprecation period (assuming there's any appetite to try; more on this later). Unlike [N2555], the parameter kind (the pattern of the parameter pack) is *also* misleading.

A naïve reading of the declaration of `F` would interpret it as a template capable of taking any number of arguments, *each of any kind*. However, `F` will bind to *any* template template argument whatsoever, irrespective of the number or kinds of template parameters or even any associated constraints on the argument template.

§ 3 Design

In the case of `apply`, what's actually desired is a form of template template parameter that does not suggest a capability that expresses our lack of knowledge about the bound template template argument. In each of the prior formulations of `apply`, the *template-head* of `F` has conveyed misleading information about the properties of `F`, when what we really want is a way to say that we don't know anything about `F`. So let's get rid of the *template-head* entirely:

```
template <template ~~<?...>~~ typename F, ?... Args>
using apply = F<Args...>; // easy peasy!
```

Now, we can actually distinguish between a template that accepts any arguments whatsoever and a template that we know nothing about. We now have the tools we need to fully express intent.

§ 3.1 Constrained template template parameters

Headless template template parameters are essentially free of *any* constraints, as they are not constrained by parameter count or kind. It seems likely that one of the first things a

programmer will want to do is to add some form of constraint on these parameters. The most obvious constraint is a check for the validity of a set of template arguments:

```
template <template typename T, ?... Args>
  concept specializable = requires { typename T<Args...>; };
```

To ease the application of constraints, a concept name denoting a template concept may be used to declare a headless template template parameter. This works exactly as it does for type constraints, except that the concept's prototype parameter must be a template template parameter rather than a type parameter.

```
template <specializable<int, 5> TT> void f();

// Same as:
//   template <template typename TT> requires specializable<TT, int, 5>
//   void f();
```

§ 4 Examples

Following are some examples copied from [P1985R3]. The examples have been updated with diff marks showing the difference in syntax with the application of headless template template parameters. In all cases, the semantics of the example are unchanged. As the difference is entirely in the absence of a *template-head*, headless template template parameters result in simpler syntax that is also more representative of intent.

```
// is_specialization_of
template <typename T, template <?...> typename Type>
constexpr bool is_specialization_of_v = false;

template <?... Params, template <?...> typename Type>
constexpr bool is_specialization_of_v<Type<Params...>, Type> = true;

template <typename T, template <?...> typename Type>
concept specialization_of = is_specialization_of_v<T, Type>;
```

```
template<typename Type, template <?...> typename Templ>
constexpr bool is_specialization_of_v = (template_of(^Type) == ^Templ);
```

```
template <?> constexpr bool is_typename_v             = false;
template <typename T> constexpr bool is_typename_v<T> = true;
template <?> constexpr bool is_value_v                = false;
template <auto V> constexpr bool is_value_v<V>        = true;
template <?> constexpr bool is_template_v             = false;
template <template <?...> typename A>
constexpr bool is_template_v<A>                       = true;

// The associated type for each trait:
template <? X> struct is_typename : std::bool_constant<is_typename_v<X>>
        {};
template <? X> struct is_value    : std::bool_constant<is_value_v<X>> {};
template <? X> struct is_template : std::bool_constant<is_template_v<X>>
        {};
```

```
template <?> struct box; // impossible to define body

template <auto X>
struct box<X> { static constexpr decltype(X) result = X; };

template <typename X>
struct box<X> { using result = X; };

template <template <?...> typename X>
struct box<X> {
  template <?... Args>
  using result = X<Args...>;
};

template <template <?> typename Map,
          template <?...> typename Reduce,
          ?... Args>
using map_reduce = Reduce<Map<Args>::result...>;

template <int... xs> using sum = box<(0 + ... + xs)>;
template <? X> using boxed_is_typename = box<is_typename_v<X>>;
static_assert(2  ==  map_reduce<boxed_is_typename,  sum,  int,  1,  long,
        std::vector>::result);

template<typename T>
struct unwrap
{
  using result = T;
};

template<typename T, T t>
struct unwrap<std::integral_constant<T, t>>
{
  static constexpr T result = t;
};

template <template <?…> typename T, typename... Params>
using apply_unwrap = T<unwrap<Params>::result...>;

apply_unwrap<std::array, int, std::integral_constant<std::size_t, 5>> arr;

template <template <?…> typename F,
          ? ... Args1>
struct curry {
  template <?... Args2>
  using func = F<Args1..., Args2...>;
};

template<template<?…> class C, typename... Ps>
auto make_unique(Ps&&... ps) {
          return     unique_ptr<decltype(C(std::forward<Ps>(ps)...))>(new
        C(std::forward<Ps>(ps)...));
}
```

§ 5 Wording

§7.5.5.2 [expr.prim.lambda.closure]/8

8 [*Note:* The function call operator or operator template can be constrained (13.5.3 [temp.constr.decl]) by a ~~*type-constraint*~~ *constraint-specifier* (13.2 [temp.param]), a *requires-clause* (13.1 [temp.pre]), or a trailing *requires-clause* (9.3 [dcl.decl]).

[*Example:*

```
template <typename T> concept C1 = /* ... */;
template <std::size_t N> concept C2 = /* ... */;
template <typename A, typename B> concept C3 = /* ... */;

auto f = []<typename T1, C1 T2> requires C2<sizeof(T1) +
        sizeof(T2)>
          (T1 a1, T1 b1, T2 a2, auto a3, auto a4) requires
        C3<decltype(a4), T2> {
   // T2 is constrained by a ~~type-constraint~~ constraint-
        specifier.
  // T1 and T2 are constrained by a requires-clause, and
   // T2 and the type of a4 are constrained by a trailing
        *requires-clause*.
};
```

— *end example*]
— *end note*]

§7.5.7.4 [expr.prim.req.compound]

compound-requirement:
{ *expression* } `noexcept`$_{opt}$ *return-type-requirement*$_{opt}$;

return-type-requirement:
-> ~~*type-constraint*~~ *constraint-specifier*

§7.5.7.4 [expr.prim.req.compound]/1

1 A *compound-requirement* asserts properties of the *expression* *E*. Substitution of template arguments (if any) and verification of semantic properties proceed in the following order:

(1.1) — Substitution of template arguments (if any) into the *expression* is performed.

(1.2) — If the `noexcept` specifier is present, *E* shall not be a potentially-throwing expression (14.5 [except.spec]).

(1.3) — If the *return-type-requirement* is present, then:

(1.3.1) — Substitution of template arguments (if any) into the *return-type-requirement* is performed.

— The *constraint-specifier* shall name a type concept.

(1.3.2) — The immediately-declared constraint (13.2 [temp.param]) of the ~~*type-constraint*~~ *constraint-specifier* for `decltype((`*E*`))` shall be satisfied.

[…]

[…]

§9.2.9.3 [dcl.type.simple]/2

2 The component names of a *simple-type-specifier* are those of its *nested-name-specifier*, *type-name*, *simple-template-id*, *template-name*, and/or ~~*type-constraint*~~ *constraint-specifier* (if it is a *placeholder-type-specifier*). The component name of a *type-name* is the first name in it.

§9.2.9.7.1 [dcl.spec.auto.general]

placeholder-type-specifier:
~~*type-constraint*~~$_{opt}$ *constraint-specifier*$_{opt}$ `auto`
~~*type-constraint*~~$_{opt}$ *constraint-specifier*$_{opt}$ `decltype ( auto )`

§9.2.9.7.1 [dcl.spec.auto.general]/2

2 A *placeholder-type-specifier* of the form ~~*type-constraint*~~$_{opt}$ *constraint-specifier*$_{opt}$ `auto` can be used as a *decl-specifier* of the *decl-specifier-seq* of a *parameter-declaration* of a function declaration or *lambda-expression* and, if it is not the `auto` *type-specifier* introducing a *trailing-return-type* (see below), is a *generic parameter type placeholder* of the function declaration or *lambda-expression*.

[…]

§9.2.9.7.2 [dcl.type.auto.deduct]/2.1

(2.1) — For a non-discarded `return` statement that occurs in a function declared with a return type that contains a placeholder type, `T` is the declared return type.

[…]

If *E* has type `void`, `T` shall be either ~~*type-constraint*$_{opt}$~~ *constraint-specifier*$_{opt}$ `decltype(auto)` or *cv* ~~*type-constraint*$_{opt}$~~ *constraint-specifier*$_{opt}$ `auto`.

§9.2.9.7.2 [dcl.type.auto.deduct]/3

3 If the *placeholder-type-specifier* is of the form ~~*type-constraint*$_{opt}$~~ *constraint-specifier*$_{opt}$ `auto`, the deduced type `T′` replacing `T` is determined using the rules for template argument deduction. If the initialization is copy-list-initialization, a declaration of `std::initializer_list` shall precede (6.5.1 [basic.lookup.general]) the *placeholder-type-specifier*. Obtain `P` from `T` by replacing the occurrences of ~~*type-constraint*$_{opt}$~~ *constraint-specifier*$_{opt}$ `auto` either with a new invented type template parameter `U` or, if the initialization is copy-list-initialization, with `std::initializer_list<U>`. Deduce a value for `U` using the rules of template argument deduction from a function call (13.10.3.2 [temp.deduct.call]), where `P` is a function template parameter type and the corresponding argument is *E*. If the deduction fails, the declaration is ill-formed. Otherwise, `T′` is obtained by substituting the deduced `U` into `P`.

[…]

§9.2.9.7.2 [dcl.type.auto.deduct]/4

4 If the *placeholder-type-specifier* is of the form ~~*type-constraint*$_{opt}$~~ *constraint-specifier*$_{opt}$ `decltype(auto)`, `T` shall be the placeholder alone. The type deduced for `T` is determined as described in 9.2.9.6 [dcl.type.decltype], as though *E* had been the operand of the `decltype`.

[…]

§9.2.9.7.2 [dcl.type.auto.deduct]/5

5 For a *placeholder-type-specifier* with a ~~*type-constraint*~~ *constraint-specifier*, the *constraint-specifier* shall name a type concept (13.7.9 [temp.concept]) and the immediately-declared constraint (13.2 [temp.param]) of the ~~*type-constraint*~~ *constraint-specifier* for the type deduced for the placeholder shall be satisfied.

§9.3.4.6 [dcl.fct]/22

22 An *abbreviated function template* is a function declaration that has one or more generic parameter type placeholders (9.2.9.7 [dcl.spec.auto]). An abbreviated function template is equivalent to a function template (13.7.7 [temp.fct]) whose *template-parameter-list* includes one invented type *template-parameter* for each generic parameter type placeholder of the function declaration, in order of appearance. For a *placeholder-type-specifier* of the form `auto`, the invented parameter is an unconstrained *type-parameter*. For a *placeholder-type-specifier* of the form ~~*type-constraint*~~ *constraint-specifier* `auto`, the invented parameter is a *type-parameter* with that ~~*type-constraint*~~ *constraint-specifier*. The invented type *template-parameter* is a template parameter pack if the corresponding *parameter-declaration* declares a function parameter pack. If the placeholder contains `decltype(auto)`, the program is ill-formed. The adjusted function parameters of an abbreviated function template are derived from the *parameter-declaration-clause* by replacing each occurrence of a placeholder with the name of the corresponding invented *template-parameter*.

[…]

§13.2 [temp.param]/1

1 The syntax for *template-parameter*s is:

template-parameter:
type-parameter
parameter-declaration

type-parameter:
type-parameter-key . . .$_{opt}$ *identifier*$_{opt}$
type-parameter-key identifier$_{opt}$ = *type-id*
~~*type-constraint*~~ *constraint-specifier* . . .$_{opt}$ *identifier*$_{opt}$
~~*type-constraint*~~ *constraint-specifier identifier*$_{opt}$ = *type-id*
template-head type-parameter-key . . .$_{opt}$ *identifier*$_{opt}$
template-head type-parameter-key identifier$_{opt}$ = *id-expression*
`template` *type-parameter-key* . . .$_{opt}$ *identifier*$_{opt}$
`template` *type-parameter-key identifier*$_{opt}$ = *id-expression*

type-parameter-key:
`class`
`typename`

~~*type-constraint*~~ *constraint-specifier*:
nested-name-specifier$_{opt}$ *concept-name*
nested-name-specifier$_{opt}$ *concept-name* < *template-argument-list*$_{opt}$ >

The component names of a ~~*type-constraint*~~ *constraint-specifier* are its *concept-name* and those of its *nested-name-specifier* (if any).

[…]

§13.2 [temp.param]/3

3 The *identifier* in a *type-parameter* is not looked up. A *type-parameter* whose *identifier* does not follow an ellipsis defines its *identifier* to be a *typedef-name* ~~(if declared without `template`)~~ or *template-name* ~~(if declared with `template`)~~ in the scope of the template declaration. The *identifier* is a *template-name* if it is declared with `template` or with a *constraint-specifier* naming a template concept (13.7.9 [temp.concept]); otherwise, it is a *typedef-name*.

[…]

§13.2 [temp.param]/4

4 A ~~*type-constraint*~~ *constraint-specifier* `Q` that designates a concept `C` can be used to constrain a contextually-determined type, template, or template ~~type~~ parameter pack `T` with a *constraint-expression* `E` defined as follows. If `Q` is of the form `C<`A_1`, ⋯, `A_n`>`, then let `E′` be `C<T, `A_1`, ⋯, `A_n`>`. Otherwise, let `E′` be `C<T>`. If `T` is not a pack, then `E` is `E′`~~`;`~~; otherwise, `E` is `(E′ && ...)`. This *constraint-expression* `E` is called the *immediately-declared constraint* of `Q` for `T`. The concept designated by a ~~*type-constraint*~~ *constraint-specifier* shall be a type concept or a template concept (13.7.9 [temp.concept]).

§13.2 [temp.param]/5

5 A *type-parameter* that starts with a ~~*type-constraint*~~ *constraint-specifier* introduces the immediately-declared constraint of the ~~*type-constraint*~~ *constraint-specifier* for the parameter.

[…]

§13.2 [temp.param]/11

11 A non-type template parameter declared with a type that contains a placeholder type with a ~~*type-constraint*~~ *constraint-specifier* introduces the immediately-declared constraint of the ~~*type-constraint*~~ *constraint-specifier* for the invented type corresponding to the placeholder (9.3.4.6 [dcl.fct]).

§13.2 [temp.param]/17

17 If a *template-parameter* is a *type-parameter* with an ellipsis prior to its optional *identifier* or is a *parameter-declaration* that declares a pack (9.3.4.6 [dcl.fct]), then the *template-parameter* is a template parameter pack (13.7.4 [temp.variadic]). A template parameter pack that is a *parameter-declaration* whose type contains one or more unexpanded packs is a pack expansion. Similarly, a template parameter pack that is a *type-parameter* with a *template-parameter-list* containing one or more unexpanded packs is a pack expansion. A ~~type~~ template parameter pack declared with a ~~*type-constraint*~~ *constraint-specifier* that contains an unexpanded parameter pack is a pack expansion. A template parameter pack that is a pack expansion shall not expand a template parameter pack declared in the same *template-parameter-list*.

[…]

§13.3 [temp.names]/7

7 A *template-id* is *valid* if the named template is a template *template-parameter* declared without a *template-head* or if

(7.1) — there are at most as many arguments as there are parameters or a parameter is a template parameter pack (13.7.4 [temp.variadic]),

(7.2) — there is an argument for each non-deducible non-pack parameter that does not have a default *template-argument*,

(7.3) — each *template-argument* matches the corresponding *template-parameter* (13.4 [temp.arg]),

(7.4) — substitution of each template argument into the following template parameters (if any) succeeds, and

(7.5) — if the *template-id* is non-dependent, the associated constraints are satisfied as specified in the next paragraph.

A *simple-template-id* shall be valid unless it names a function template specialization (13.10.3 [temp.deduct]).

[…]

§13.4.4 [temp.arg.template]/3

3 A *template-argument* A matches a template *template-parameter* P when P is declared without a *template-head*, A is the name of a template *template-parameter* declared without a *template-head*, or when P is at least as specialized as ~~the *template-argument*~~ A, ignoring constraints on A if P is unconstrained. ~~In this comparison, if P is unconstrained, the constraints on A are not considered.~~ If P ~~contains~~ is declared with a *template-head* containing a template parameter pack (13.7.4 [temp.variadic]), then A also matches P if each of A's template parameters matches the corresponding template parameter in the *template-head* of P (this behavior is deprecated; see D.# [depr.temp.match]). Two template parameters match if they are of the same kind (type, non-type, template), for non-type *template-parameter*s, their types are equivalent (13.7.7.2 [temp.over.link]), and for template *template-parameter*s, each of their corresponding *template-parameter*s matches, recursively. ~~When P's *template-head* contains a template parameter pack (13.7.4 [temp.variadic]), the template parameter pack~~ A template parameter pack in the *template-head* of P will match zero or more template parameters or template parameter packs in the *template-head* of A with the same type and form as the template parameter pack in P (ignoring whether those template parameters are template parameter packs).

[*Example:*

```
template<class T> class A { /* ... */ };
template<class T, class U = T> class B { /* ... */ };
template<class ... Types> class C { /* ... */ };
template<auto n> class D { /* ... */ };
template<template class P> class W { /* ... */ };
template<template<class> class P Q> class X { /* ... */ };
template<template<class ...> class Q R> class Y { /* ... */
        };
template<template<int> class R S> class Z { /* ... */ };

W<A> wa;              // OK
W<B> wb;              // OK
W<C> wc;              // OK
W<D> wd;              // OK
X<A> xa;              // OK
X<B> xb;              // OK
X<C> xc;              // OK
Y<A> ya;              // OK
Y<B> yb;              // OK
Y<C> yc;              // OK
Z<D> zd;              // OK
```

— end example]

[*Example:*

```
template <class T> struct eval X {_};

template <template <class, class...> class TT, class T1,
        class... Rest>
struct eval<TT<T1, Rest...>> { };
using eval1 = TT<T1, Rest...>;

template <template class TT, class T1, class... Rest>
using eval2 = TT<T1, Rest...>;

template <class T1> struct A;
template <class T1, class T2> struct B;
template <int N> struct C;
template <class T1, int N> struct D;
template <class T1, class T2, int N = 17> struct E;

X<eval1<A<,   int>>   e1A;                              //
        OK, matches partial specialization of eval
X<eval1<B<,     int,     float>>     e1B;               //
        OK, matches partial specialization of eval
X<eval1<C<, 17>> e1C;            // error: C does not match
        TT in partial specialization eval1
X<eval1<D<, int, 17>> e1D;       // error: D does not match
        TT in partial specialization eval1
X<eval1<E<, int, float>> e1E;    // error: E does not match
        TT in partial specialization eval1

X<eval2<A, int>> e2A;            // OK
X<eval2<B, int, float>> e2B;     // OK
X<eval2<C,  17>>  e2C;                      // error: non-
        type argument provided for T1 in eval2
X<eval2<D,  int,  17>>  e2D;                // error: non-
        type argument provided for Rest in eval2
X<eval2<E, int, float>> e2E;     // OK
```

— end example]

[*Example:*

```
template<typename T> concept C = requires (T t) { t.f(); };
template<typename T> concept D = C<T> && requires (T t) {
        t.g(); };

template<template<C> class P> struct S { };

template<C> struct X { };
template<D> struct Y { };
template<typename T> struct Z { };

S<X> s1;                 // OK, X and P have equivalent
        constraints
```

```
S<Y> s2;                  // error: P is not at least as
        specialized as Y
S<Z> s3;             // OK, P is at least as specialized as
        Z
```

— *end example*]

§13.5.3 [temp.constr.decl]/2

2 Constraints can also be associated with a declaration through the use of ~~*type-constraints*~~ *constraint-specifiers* in a *template-parameter-list* or *parameter-type-list*. Each of these forms introduces additional *constraint-expression*s that are used to constrain the declaration.

§13.5.3 [temp.constr.decl]/3.3

(3.3) — Otherwise, the associated constraints are the normal form of a logical AND expression (7.6.14 [expr.log.and]) whose operands are in the following order:

(3.3.1) — the *constraint-expression* introduced by each ~~*type-constraint*~~ *constraint-specifier* (13.2 [temp.param]) in the declaration's *template-parameter-list*, in order of appearance, and

(3.3.2) — the *constraint-expression* introduced by a *requires-clause* following a *template-parameter-list* (13.1 [temp.pre]), and

(3.3.3) — the *constraint-expression* introduced by each ~~*type-constraint*~~ *constraint-specifier* in the parameter-type-list of a function declaration, and

(3.3.4) — the *constraint-expression* introduced by a trailing *requires-clause* (9.3 [dcl.decl]) of a function declaration (9.3.4.6 [dcl.fct]).

§13.7.1 [temp.decls.general]/3

3 For purposes of name lookup and instantiation, default arguments, ~~*type-constraints*~~ *constraint-specifiers*, *requires-clause*s (13.1 [temp.pre]), and *noexcept-specifier*s of function templates and of member functions of class templates are considered definitions; each default argument, ~~*type-constraint*~~ *constraint-specifier*, *requires-clause*, or *noexcept-specifier* is a separate definition which is unrelated to the templated function definition or to any other default arguments, ~~*type-constraints*~~ *constraint-specifiers*, *requires-clause*s, or *noexcept-specifier*s. For the purpose of instantiation, the substatements of a constexpr if statement (8.5.2 [stmt.if]) are considered definitions.

§13.7.7.2 [temp.over.link]/6

6 Two *template-head*s are *equivalent* if their *template-parameter-list*s have the same length, corresponding *template-parameter*s are equivalent and are both declared with ~~*type-constraints*~~ *constraint-specifiers* that are equivalent if either *template-parameter* is declared with a ~~*type-constraint*~~ *constraint-specifier*, and if either *template-head* has a *requires-clause*, they both have *requires-clause*s and the corresponding *constraint-expression*s are equivalent. Two *template-parameter*s are *equivalent* under the following conditions:

(6.1) — they declare template parameters of the same kind,

(6.2) — if either declares a template parameter pack, they both do,

(6.3) — if they declare non-type template parameters, they have equivalent types ignoring the use of ~~*type-constraints*~~ *constraint-specifier*s for placeholder types, and

(6.4) — if they declare template template parameters, either neither has a *template-head* or they both do and their ~~template parameters~~ *template-head*s are equivalent.

When determining whether types or ~~*type-constraints*~~ *constraint-specifiers* are equivalent, the rules above are used to compare expressions involving template parameters. Two *template-head*s are *functionally equivalent* if they accept and are satisfied by (13.5.2 [temp.constr.constr]) the same set of template argument lists.

§13.7.7.3 [temp.func.order]/3

3 To produce the transformed template, for each type, non-type, or template template parameter (including template parameter packs (13.7.4 [temp.variadic]) thereof) synthesize a unique type, value, or class template respectively and substitute it for each occurrence of that parameter in the function type of the template~~.~~:

(3.1) — For a non-type template parameter, the type of the synthesized value is the type of the parameter after substitution of prior template parameters.

[*Note:* The type replacing the placeholder in the type of the value synthesized for a non-type template parameter is also a unique synthesized type. — *end note*]

(3.2) — For a template template parameter declared with a *template-head*, the *template-head* of the synthesized class template is the result of

substitution into the *template-head* of the template template parameter. For a template template parameter declared without a *template-head*, the synthesized class template does not match (13.4.4 [temp.arg.template]) any template *template-parameter* declared with a *template-head*.

[…]

§13.7.9 [temp.concept]/2

2 A *concept-definition* declares a concept. Its *identifier* becomes a *concept-name* referring to that concept within its scope. The optional *attribute-specifier-seq* appertains to the concept.

[*Example:*

```
template<typename T>
concept C = requires(T x) {
  { x == x } -> std::convertible_to<bool>;
};

template<typename T>
   requires C<T>      // C constrains f1(T) in constraint-
         expression
T f1(T x) { return x; }

template<C T>        // C, as a ~~type constraint~~ constraint-
         specifier, constrains f2(T)
T f2(T x) { return x; }
```

— *end example*]

§13.7.9 [temp.concept]/7

7 The first declared template parameter of a concept definition is its *prototype parameter*. A *type concept* is a concept whose prototype parameter is a type *template-parameter*. A *template concept* is a concept whose prototype parameter is a template *template-parameter*.

§13.9.2 [temp.inst]/2

2 *[…]*

[*Note:* Within a template declaration, a local class (11.6 [class.local]) or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, *noexcept-specifier*s, and non-static data member initializers, if

any, but not their ~~*type-constraints*~~ *constraint-specifiers* or *requires-clauses*). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated as part of the instantiation of the entity within which the local class or enumeration is declared. — *end note*]

§13.9.2 [temp.inst]/17

17 The ~~*type-constraints*~~ *constraint-specifiers* and *requires-clause* of a template specialization or member function are not instantiated along with the specialization or function itself, even for a member function of a local class; substitution into the atomic constraints formed from them is instead performed as specified in 13.5.3 [temp.constr.decl] and 13.5.2.3 [temp.constr.atomic] when determining whether the constraints are satisfied or as specified in 13.5.3 [temp.constr.decl] when comparing declarations.

§D [depr]

§ **D.# Matching template *template-parameter*s with parameter packs [depr.temp.match]**

1 A template parameter pack in the *template-head* of a template *template-parameter* is permitted to match zero or more template parameters in the *template-head* of a template *template-argument* (13.4.4 [temp.arg.template]). This behavior is deprecated.

[*Example:*

```
template <template <class... Ts> class TT> class X { };
template <class A, class B> class Y;

X<Y> x;        // deprecated, TT matches A and B

template <template <class A, class B> class TT> class P {
        };
template <class... Ts> class Q;

P<Q> p;        // OK
```

— *end example*]

§ 6 References

[N2555] D. Gregor, E. Niebler. 2008-02-29. Extending Variadic Template Template Parameters (Revision 1).
https://wg21.link/n2555

[P0522R0] James Touton, Hubert Tong. 2016-11-11. DR: Matching of template template-arguments excludes compatible templates.

https://wg21.link/p0522r0

[P1985R3] Gašper Ažman, Mateusz Pusz, Colin MacLean, Bengt Gustafsonn, Corentin Jabot. 2022-09-17. Universal template parameters.

https://wg21.link/p1985r3

[P2989R0] Corentin Jabot, Gašper Ažman. 2023-10-14. A Simple Approach to Universal Template Parameters.

https://wg21.link/p2989r0